
Lecture Assist: An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

By

Iftikhar Ahmed — ID # 2121019642

Salman Shafi — ID # 2211386042

Jarinah Tasnim — ID # 2121026642

Supervised by

Dr. Adnan Areefen

Department of Electrical and Computer Engineering

North South University

Submitted in partial fulfillment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

July 26, 2026



DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING
NORTH SOUTH UNIVERSITY

APPROVAL

Iftikhar Ahmed , Salman Shafi , and Jarinah Tasnim
from the Department of Electrical and Computer Engineering, North South University
have worked on the Senior Design Project titled:

**“LectureAssist: An Agentic, Multimodal AI Assistant that
Turns Lectures into Adaptive Study Material”**

under the supervision of **Dr. Adnan Areefen** in partial fulfillment of the requirement
for the degree of Bachelor of Science in Computer Science and Engineering and has been
accepted as satisfactory.

Supervisor’s Signature

.....

Dr. Adnan Areefen

Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

Chairman’s Signature

.....

Dr. Mohammad Abdul Matin

Professor
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

DECLARATION

It is to declare that this project is our original work. No part of this work has been submitted elsewhere, partially or entirely, for the award of any other degree or diploma.

All project related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor.

Relevant previous works presented in this report have been adequately acknowledged and cited. The plagiarism policy, as stated by the supervisor, has been maintained.

Students' Names & Signatures

1. **Iftikhar Ahmed**

2. **Salman Shafi**

3. **Jarinah Tasnim**

Acknowledgements

This work would not have been possible without the input and support of many people over the last two trimesters. We would like to express our gratitude to everyone who contributed to it in some way or other.

First and foremost, we would like to thank our supervisor, **Dr. Adnan Areefen**, Department of Electrical and Computer Engineering, North South University, for his guidance throughout this project. His insistence that the evaluation be made rigorous that the generation prompts be shown rather than described, that chain-of-thought and program-of-thought baselines be compared directly against our agentic pipeline, and that question generation and answer generation be treated as two separate agents whose answers must be cross-checked fundamentally shaped this work, and turned it from a demonstration into a study.

Our sincere gratitude goes to the Department of Electrical and Computer Engineering, North South University, for providing the academic environment and the resources that made this project possible.

We are also thankful to the open-source community, whose freely available models and tools — Gemma 3, Ollama, Whisper, EasyOCR, Sentence-Transformers and FAISS — are the foundation on which this system is built, and without which a project of this scope could not have been undertaken without significant funding.

Last but not the least, we owe a great deal to our families, including our parents, for their unconditional love and immense emotional support.

Abstract

Recorded lectures and digital course material are now abundant, but the tools that help students convert that material into practice and self assessment are not. Existing study aids either require the student to paste in clean text by hand, or depend on paid cloud APIs that are impractical in bandwidth limited and cost-sensitive settings. This report presents **LectureAssist**, an end-to-end agentic AI system that turns a raw lecture video or an academic document into structured study and assessment material without any proprietary cloud service. The system couples a dual-channel extraction layer scene-adaptive optical character recognition that distinguishes slides, blackboards, whiteboards and on-screen text, combined with Whisper-based speech transcription to a multi-agent generation pipeline that plans, retrieves, generates, validates and refines five types of quiz questions, all served by locally hosted Gemma 3 models through the Ollama runtime on a single consumer-grade GPU. A retrieval augmented chat interface and an adaptive difficulty engine that tracks per topic performance complete the loop from raw media to personalised assessment. The system was evaluated on a large scale study in which multiple-choice questions were generated from a college-level calculus chapter across three difficulty levels, and scored by an independent, larger judge model on question quality, grounding, diversity and separately — *answer correctness*, using a second Answer Generator agent that re-solves each question without seeing the marked option. The work demonstrates that high quality, personalised educational AI is achievable on accessible hardware, and that question generators must be judged on answer correctness and diversity, not on fluency alone.

Table of Contents

Table of Contents	iv
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Background and Motivation	1
1.2 Purpose and Goal of the Project	2
1.3 Organization of the Report	3
2 Background and Literature Review	4
2.1 Preliminaries	4
2.1.1 Large Language Models and Prompting	5
2.1.2 Agent Based Architectures	6
2.1.3 Evaluation of Educational Question Generation	7
2.2 Literature Review	9
2.2.1 Evolution of Educational Question Generation	9
2.2.2 Large Language Model Based Educational Question Generation	10
2.2.3 Evaluation Frameworks for Educational Question Generation	12
2.3 Research Gap	13
3 Methodology	15
3.1 System Design	15
3.1.1 Data flow	15
3.1.2 Scene-adaptive extraction	16
3.1.3 Retrieval	16
3.2 Hardware and/or Software Components	16
3.3 Hardware and/or Software Implementation	17
3.3.1 The agentic question-generation loop	17
3.3.2 The four generation pipelines	18
3.3.3 Answer correctness: two agents that must agree	20
3.3.4 Multi-tier grading and adaptation	20

4	Investigation/Experiment, Result, Analysis and Discussion	22
4.1	Environment Setup	22
4.2	Evaluation Protocol	23
4.3	Results and Discussion	25
4.3.1	What EQGBench does	25
4.3.2	What ReQUESTA does	25
4.3.3	How this fits our project	26
4.3.4	How our evaluation works	26
4.3.5	Our results	27
4.4	Summary	28
	References	28

List of Figures

List of Tables

3.1	Software components, alternatives, and selection rationale.	17
4.1	ReQUESTA results on evaluation metrics comparable to this work.	26
4.2	Average complexity metrics across different pipelines.	27
4.3	Comparison between DeepSeek-R1 (EQGBench Mathematics subset) and our proposed pipelines. Higher values indicate better performance.	27
4.4	Diversity, grounding and answer correctness by difficulty and pipeline. . . .	27
4.5	Comparison between the published ReQUESTA results and the proposed framework. The values are reported using each study’s original evaluation methodology and are presented for reference only.	28

Chapter 1

Introduction

This chapter motivates the project, states its purpose and contributions, and outlines the structure of the remainder of the report.

1.1 Background and Motivation

The proliferation of recorded lectures and digital course material has created a fundamental imbalance in self directed learning: students have more content than ever, but fewer tools with which to extract, consolidate and assess their understanding of it. A single semester course now routinely produces dozens of hours of recorded lectures, slide decks and reference chapters. Reviewing that material passively re-watching a video, re-reading a chapter is one of the least effective study strategies available. Decades of work in the learning sciences instead point to *retrieval practice*: repeatedly testing oneself on the material produces substantially more durable learning than re-exposure to it. The obstacle is supply. Producing good practice questions is slow, expert work, and the questions that do exist are concentrated at the end of textbook chapters, are not tied to what a particular instructor actually emphasised in class, and run out long before a student has mastered a weak topic.

Existing AI assisted study tools address this gap only partially. Text based chatbots require the student to manually transcribe or paste lecture content before they can help, which puts the burden of the hardest step getting the lecture out of the video and into text back on the student. Cloud dependent question generation services remove that burden, but they are inaccessible in bandwidth limited regions, they charge per token at a scale that makes routine practice expensive, and they require sending course material which may be copyrighted or institutionally restricted to a third party server. Crucially, none of these tools close the full loop from raw instructional media, through intelligent content understanding, to graded and personalised assessment without human intervention at each stage.

There is also a quality problem that is easy to overlook. A large language model asked to write a multiple choice question will almost always produce something that *reads* like

a good question. Whether the option it marks as correct actually *is* correct is a separate matter, and one that fluency based evaluation cannot detect. A study tool that confidently teaches a student the wrong answer is worse than no tool at all. Any serious system for automatic question generation therefore has to be evaluated on answer correctness, and on whether it keeps producing genuinely different questions rather than paraphrasing the same one, not merely on whether its output is well written.

1.2 Purpose and Goal of the Project

The goal of this project is to build and rigorously evaluate **LectureAssist**, a fully local, API free system that ingests a lecture video or an academic document and autonomously produces validated, difficulty controlled assessment material, together with a retrieval grounded chat interface over the same content.

The system is built around three ideas working in concert. First, a *dualchannel extraction* layer simultaneously processes the visual channel of a lecture using scene type adaptive OCR that distinguishes slides, blackboards, whiteboards and on-screen text, each of which needs different image preprocessing and its audio channel via Faster Whisper transcription, producing a unified, timestamped lecture timeline. Second, a *multi-agent generation pipeline* decomposes quiz creation into specialist agents for planning, retrieval, generation, validation and targeted refinement, enabling structured quality control at the level of the individual question. Third, an *adaptive difficulty engine* maintains per topic performance histories across sessions and steers question allocation toward a student’s weak areas. The entire system runs on locally hosted Gemma 3 models through the Ollama runtime, requires no external API keys, and is packaged for single GPU deployment on Google Colab with Google Drive session persistence.

The key contributions of this work are:

- A unified multi modal lecture understanding pipeline that fuses scene adaptive OCR with filtered speech transcription into a synchronised content timeline, supporting both video and document inputs.
- A multi-agent quiz architecture implementing a Plan → Retrieve → Generate → Validate → Refine cycle across five question types (MCQ, True/False, Fill-in-the-Blank, Short Answer and Flashcard), with per-question difficulty, grounding and instruction-compliance validation, and a validator that chooses *how* to remediate a rejected question rather than merely rejecting it.
- An adaptive assessment engine that tracks per-topic student performance across sessions and personalises question difficulty and distribution without manual configuration.
- A multi-tier grading system combining deterministic matching, fuzzy string similarity and LLM-based rubric evaluation for free-text short answers, with targeted

concept-level feedback on incorrect responses.

- A fully local, API-free deployment on consumer hardware using the Ollama inference runtime, with RAG-enabled semantic chat and persistent session storage.
- A large-scale evaluation harness that compares the agentic pipeline against three direct-prompting baselines — plain, chain-of-thought and program-of-thought — and which, unlike prior automatic question generation evaluations, separates *question quality* from *answer correctness* by having a second, larger Answer Generator agent independently re-solve every generated question without seeing the marked option.

1.3 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 provides the background needed to follow the rest of the report — large language models and the prompting strategies used as baselines, retrieval-augmented generation, agentic architectures, speech recognition and OCR — then reviews the research literature and the existing commercial applications closest to this work, and identifies the gaps this project addresses. Chapter 3 describes the system design: the extraction layer, the agent architecture, the four question-generation pipelines with their prompts given verbatim, the two-agent answer cross-check, and the software components used to build it. Chapter 4 presents the evaluation: the environment, the metrics and exactly how each is computed, the large-scale multiple-choice results, and a discussion of what they show — including a failure mode of the agentic pipeline that the evaluation exposed. Chapter ?? discusses the societal, ethical and environmental implications and the design constraints they impose. Chapter ?? presents the project plan and budget. Chapter ?? maps the work onto the complex engineering problem and activity attributes. Chapter ?? summarises the project, states its limitations honestly, and sets out the work that follows from it.

Chapter 2

Background and Literature Review

This chapter presents the theoretical foundation and reviews the existing literature related to large language model based educational question generation. Recent advances in large language models have significantly improved the ability of intelligent systems to generate educational content, particularly multiple choice questions that support teaching, learning, and assessment. At the same time, ensuring that generated questions are pedagogically meaningful, cognitively appropriate, and aligned with educational objectives remains a challenging research problem. Consequently, recent studies have shifted their focus from simply generating fluent questions toward developing structured generation frameworks and comprehensive evaluation methodologies.

This chapter first introduces the fundamental concepts required to understand the proposed system, including large language models, prompting techniques, agent based architectures, and evaluation strategies for educational question generation. It then reviews recent research that investigates controlled question generation using large language models, with particular emphasis on hybrid multi agent generation frameworks and educational question evaluation benchmarks. Existing educational applications are subsequently discussed before identifying the research gaps that motivate the proposed work presented in this thesis.

2.1 Preliminaries

Educational question generation using large language models integrates concepts from natural language processing, prompt engineering, intelligent agent systems, and educational assessment. Understanding these foundational concepts is essential for designing systems that generate high quality assessment items while maintaining pedagogical relevance and consistency. This section introduces the key technologies and methodologies that underpin the proposed framework, including large language models, prompting strategies, agent based architectures, and evaluation techniques. These concepts provide the theoretical

foundation for the methodology presented in later chapters.

2.1.1 Large Language Models and Prompting

Large Language Models (LLMs) have emerged as one of the most significant advancements in artificial intelligence, demonstrating remarkable capabilities in understanding, reasoning over, and generating natural language. Built upon the Transformer architecture, LLMs are pretrained on extensive text corpora using self supervised learning, enabling them to capture complex linguistic patterns, semantic relationships, and contextual information. Through subsequent instruction tuning and alignment techniques, these models can perform a wide variety of language tasks including summarization, translation, question answering, content generation, and educational assistance without requiring task specific model architectures. Their versatility has led to widespread adoption across educational, industrial, and research applications, particularly in tasks that require generating coherent and contextually relevant textual content. [1, 2]

The emergence of LLMs has fundamentally changed the landscape of automatic educational question generation. Earlier approaches often relied on handcrafted linguistic rules or supervised neural models trained for a single task, limiting their adaptability across different educational domains. In contrast, modern LLMs can generate diverse assessment items from instructional materials using natural language instructions alone. This capability allows a single model to generate questions of different formats, difficulty levels, and cognitive requirements without extensive retraining. Consequently, recent research has increasingly focused on designing effective interaction strategies that enable language models to generate pedagogically meaningful and educationally appropriate assessment items rather than developing specialized generation models for each individual task. [1, 2]

The quality of responses produced by an LLM depends not only on the model itself but also on how the task is presented. Prompt engineering refers to the process of designing structured instructions that guide the model toward producing outputs aligned with specific objectives. In educational question generation, prompts commonly specify the source material, target knowledge concepts, question type, difficulty level, and expected output format. Well designed prompts reduce ambiguity, improve consistency, and help ensure that generated questions remain relevant to the learning objectives. Recent systems further enhance this process by dynamically constructing prompts using contextual information generated during intermediate processing stages, allowing the model to produce more accurate and context aware assessment items. [1, 2]

Several prompting techniques have been proposed to further improve educational question generation. One widely adopted approach is Chain of Thought prompting, which encourages the model to explicitly perform intermediate reasoning before generating the final output. Rather than producing a question immediately, the model first analyzes the instructional content, identifies important concepts, determines appropriate learning objectives, and then constructs a question that reflects the intended educational outcome.

This reasoning process improves the generation of higher order questions requiring conceptual understanding or logical inference while reducing inconsistencies in the generated assessment items. Modern educational generation frameworks frequently employ Chain of Thought prompting during planning and evaluation stages to improve both reliability and pedagogical quality. [2]

Another important prompting strategy is persona based prompting, where the language model is instructed to assume the role of an experienced teacher, curriculum designer, or assessment specialist. Assigning a professional role encourages the model to generate questions that more closely resemble those created by human educators while adhering to established educational practices. Recent studies also incorporate self critique prompting, which introduces an internal evaluation stage after question generation. During this stage, the language model reviews its own output by examining question clarity, answer correctness, distractor quality, and alignment with instructional objectives before revising the question if necessary. Together, these prompting strategies improve the controllability, consistency, and educational quality of modern large language model based question generation systems. [2]

2.1.2 Agent Based Architectures

Early large language model based educational question generation systems primarily relied on a single prompt to transform instructional content into assessment questions. Although these systems demonstrated impressive language generation capabilities, they often struggled to maintain consistency, accurately control cognitive complexity, and verify the correctness of generated questions. A single language model was expected to understand the source material, identify important concepts, generate meaningful questions, construct plausible distractors, and evaluate the quality of the final output within a single interaction. As the complexity of educational tasks increased, this monolithic approach frequently produced questions with ambiguous wording, incorrect answers, or distractors that failed to effectively distinguish different levels of student understanding. Consequently, recent research has shifted towards agent based architectures that decompose the overall generation process into multiple coordinated subtasks.

An agent based architecture consists of several specialized agents that collaborate to accomplish a complex task. Rather than assigning every responsibility to a single language model interaction, each agent performs a dedicated function while exchanging information with other agents throughout the workflow. Individual agents may be responsible for analyzing instructional content, identifying learning objectives, generating questions, evaluating assessment quality, or formatting the final output. This modular design improves transparency, allows intermediate outputs to be verified before proceeding to subsequent stages, and enables individual components to be refined independently without affecting the entire system. Such characteristics make agent based architectures particularly suitable for educational applications where accuracy, consistency, and pedagogical quality are

essential.

A representative example of this approach is the ReQUESTA framework, which employs a hybrid multi agent architecture for automatic multiple choice question generation. Instead of relying solely on a language model, the framework combines deterministic rule based components with LLM powered reasoning agents. Rule based modules perform structured operations such as document segmentation, workflow coordination, and output formatting, while specialized language model agents perform higher level reasoning tasks including concept identification, question generation, answer verification, and quality evaluation. This hybrid design leverages the reliability of traditional algorithms together with the flexibility and reasoning capability of modern large language models, resulting in more robust educational question generation.

Within the ReQUESTA workflow, the instructional material is first divided into manageable text segments before being analyzed by a planning agent. The planning agent identifies important concepts, determines suitable learning objectives, and prepares a structured generation plan for subsequent stages. Based on this plan, specialized generation agents create different categories of questions that target various cognitive skills. A dedicated evaluation agent then reviews each generated question by examining its clarity, correctness, and educational relevance. If deficiencies are identified, the question is returned to the corresponding generation agent for revision, creating an iterative refinement process that continues until the generated assessment satisfies predefined quality requirements. Finally, formatting components standardize the presentation of the questions for learners.

The adoption of agent based architectures demonstrates that improvements in educational question generation depend not only on increasingly capable language models but also on carefully designed workflows that coordinate reasoning, generation, and evaluation. By separating complex educational tasks into specialized components, these architectures provide greater control over question quality, improve alignment with learning objectives, and facilitate the development of scalable educational assessment systems. Consequently, agent based workflows have become an important direction in recent research on large language model based educational question generation.

2.1.3 Evaluation of Educational Question Generation

Evaluating automatically generated educational questions is considerably more challenging than evaluating general text generation tasks. While traditional natural language generation metrics such as BLEU, ROUGE, and METEOR measure lexical similarity between generated text and reference answers, they often fail to capture the educational quality of assessment items. Multiple valid questions can be generated from the same instructional material using different wording, reasoning processes, or assessment formats while remaining equally effective for learning. Consequently, relying solely on text similarity metrics may incorrectly penalize high quality questions that differ from reference

examples, making conventional evaluation methods insufficient for educational question generation.

Recent research has therefore shifted towards pedagogically oriented evaluation frameworks that assess whether generated questions satisfy educational objectives rather than merely matching reference text. Important evaluation criteria include the correctness of the generated answer, alignment between the question and the intended knowledge point, appropriateness of the question type, clarity of language, cognitive complexity, and the quality of distractors in multiple choice questions. These dimensions provide a more comprehensive assessment of educational usefulness by considering both linguistic quality and instructional effectiveness.

One notable contribution in this area is EQGBench, a benchmark specifically developed for evaluating educational question generation using large language models. Unlike conventional benchmarks, EQGBench introduces a multidimensional evaluation framework that measures educational quality from several complementary perspectives. The benchmark evaluates knowledge point alignment, question type alignment, overall question quality, solution quality, and competence oriented guidance. Together, these dimensions assess whether generated questions accurately reflect instructional content, employ appropriate assessment formats, provide correct and complete solutions, and promote meaningful learning outcomes rather than simple factual recall. This multidimensional framework offers a more reliable assessment of educational question generation than traditional text similarity metrics alone. [1]

Another important advancement is the use of large language models as evaluators. Instead of depending exclusively on automatic similarity metrics or manual expert review, recent systems employ language models to assess generated questions according to predefined educational criteria. Acting as impartial evaluators, these models examine question clarity, factual correctness, relevance to the source material, distractor plausibility, and overall pedagogical quality. This approach enables scalable evaluation while maintaining consistency across large numbers of generated assessment items. Human expert evaluation remains the gold standard for educational assessment; however, LLM based evaluation has emerged as a practical alternative for benchmarking and iterative system development.

The increasing emphasis on comprehensive evaluation reflects the growing maturity of educational question generation research. As generation quality continues to improve through advances in large language models and agent based workflows, evaluation methodologies have likewise evolved to measure educational effectiveness rather than linguistic similarity alone. These developments provide a stronger foundation for designing intelligent assessment systems capable of generating reliable, pedagogically meaningful, and cognitively appropriate questions for diverse educational settings. [1, 2]

2.2 Literature Review

Automatic educational question generation has been an active area of research for several decades, evolving alongside advances in natural language processing and artificial intelligence. Early approaches relied primarily on manually designed linguistic rules and handcrafted templates to transform declarative sentences into questions. Although these methods were computationally efficient and generated grammatically correct questions within restricted domains, they required extensive human effort and exhibited limited adaptability to diverse educational materials. As machine learning techniques matured, researchers gradually shifted towards data driven approaches capable of learning question generation patterns directly from large annotated datasets. More recently, the emergence of large language models has fundamentally transformed the field by enabling the generation of high quality, context aware, and pedagogically meaningful assessment items through natural language instructions rather than task specific model architectures.

Despite these advances, producing educational questions that accurately assess learning outcomes remains a challenging problem. High quality assessment items must not only be grammatically correct but also align with instructional objectives, target appropriate cognitive levels, contain unambiguous answers, and provide effective distractors for multiple choice questions. Consequently, recent research has increasingly focused on developing structured generation frameworks and comprehensive evaluation methodologies that improve both the quality and educational value of automatically generated questions.

This section reviews the major developments in educational question generation, beginning with traditional approaches before discussing recent large language model based methods. Particular emphasis is placed on two representative studies that address complementary aspects of the problem: the ReQUESTA framework, which proposes a hybrid multi agent architecture for controlled multiple choice question generation, and EQG-Bench, which introduces a comprehensive benchmark for evaluating the educational quality of questions generated by large language models. Together, these studies represent important advancements in both the generation and evaluation of intelligent educational assessment systems.

2.2.1 Evolution of Educational Question Generation

The development of automatic educational question generation has closely followed the evolution of natural language processing techniques. Early research primarily relied on rule based systems that transformed declarative sentences into interrogative forms using handcrafted linguistic rules, syntactic parsing, and manually designed templates. These approaches generated grammatically correct questions within restricted domains and offered high interpretability because the generation process was explicitly defined by human experts. However, developing and maintaining large collections of language rules required substantial manual effort, while the resulting systems often struggled to generalize across different subjects, writing styles, and educational contexts. As a result, rule based meth-

ods exhibited limited scalability and were unsuitable for generating diverse assessment items from complex instructional materials.

The availability of large educational datasets and advances in machine learning led researchers to adopt data driven approaches for question generation. Sequence-to-sequence neural networks enabled models to learn mappings between instructional content and corresponding questions directly from annotated examples rather than relying on manually designed rules. Attention mechanisms further improved these models by allowing them to focus on the most relevant portions of the input text during generation. Compared with traditional rule based systems, neural approaches produced more fluent and diverse questions while requiring significantly less manual feature engineering. Nevertheless, their performance depended heavily on the availability of large, high quality training datasets and they often struggled to generate questions requiring complex reasoning or deep conceptual understanding.

The introduction of Transformer based architectures marked another major advancement in automatic question generation. Transformer models demonstrated superior contextual understanding through self attention mechanisms, enabling them to capture long range dependencies and semantic relationships more effectively than recurrent neural networks. Pretrained language models such as BERT, T5, and BART further improved generation quality by leveraging knowledge acquired from large scale pretraining before being adapted to educational tasks. These models significantly enhanced grammatical fluency, contextual relevance, and the diversity of generated questions. However, most Transformer based systems still required task specific fine tuning and were generally designed for fixed question generation objectives, limiting their flexibility across different educational settings.

Recent progress in large language models has fundamentally transformed educational question generation. Unlike earlier approaches that required dedicated training for individual tasks, modern LLMs can generate assessment items through carefully designed natural language prompts without extensive task specific optimization. This capability enables a single model to produce multiple question formats, adjust difficulty levels, and incorporate reasoning processes using prompt engineering techniques alone. Contemporary research has therefore shifted its emphasis from improving language generation itself toward developing structured workflows, agent based architectures, and comprehensive evaluation methodologies that ensure generated questions satisfy pedagogical objectives. These developments represent a transition from purely language focused generation toward intelligent educational assessment systems capable of producing reliable, context aware, and pedagogically meaningful questions across diverse learning environments. [1, 2]

2.2.2 Large Language Model Based Educational Question Generation

The emergence of large language models has significantly advanced the field of educational question generation by enabling the production of coherent, context aware, and pedagog-

ically relevant assessment items without extensive task specific training. Unlike earlier neural approaches that required fine tuning for individual datasets or question types, modern LLMs leverage knowledge acquired during large scale pretraining to perform educational tasks through carefully designed natural language prompts. This capability has greatly improved the adaptability of question generation systems, allowing them to operate across multiple academic disciplines and educational settings while requiring only minimal modifications to the input instructions.

Recent research has increasingly focused on improving the controllability of question generation rather than solely enhancing linguistic quality. One representative study is the ReQUESTA framework, which proposes a hybrid multi agent architecture for generating cognitively diverse multiple choice questions. The framework recognizes that educational assessment involves several interconnected tasks, including identifying learning objectives, selecting appropriate concepts, constructing meaningful questions, generating plausible distractors, and verifying the quality of the final assessment item. Instead of assigning all these responsibilities to a single language model interaction, ReQUESTA distributes them among multiple specialized agents that collaborate throughout the generation process. This modular design enables greater control over each stage of question generation while reducing errors that commonly occur in single prompt systems. [2]

The ReQUESTA framework combines deterministic rule based processing with the reasoning capability of large language models. Rule based components manage operations such as document segmentation, workflow coordination, and output formatting, while specialized LLM agents perform higher level reasoning tasks including concept identification, question generation, quality evaluation, and revision. The workflow begins by dividing instructional material into manageable text segments, after which a planning agent analyzes each segment to identify key concepts and determine suitable learning objectives. Based on this analysis, dedicated generation agents create multiple choice questions targeting different cognitive skills before an evaluation agent reviews each question for correctness, clarity, and educational relevance. Questions that do not satisfy predefined quality criteria are automatically revised through an iterative feedback process until acceptable quality is achieved. This structured workflow improves both the reliability and consistency of generated assessment items compared with conventional single prompt approaches. [2]

Another important contribution of ReQUESTA is its use of advanced prompting strategies throughout the generation pipeline. The framework incorporates Chain of Thought reasoning to encourage systematic analysis before question generation, persona based prompting to emulate experienced educators during assessment design, and self critique mechanisms that enable the language model to evaluate and refine its own outputs. By integrating these techniques within a coordinated multi agent workflow, the framework produces questions with clearer wording, more accurate answers, and higher quality distractors. Experimental results reported in the study demonstrate that this hybrid architecture consistently outperforms conventional zero shot prompting approaches in both automated evaluation and expert assessment, highlighting the importance of structured

workflows in educational question generation. [2]

The findings of the ReQUESTA study demonstrate that advances in educational question generation depend not only on increasingly capable language models but also on carefully designed system architectures that coordinate planning, reasoning, generation, and evaluation. These observations have influenced recent research by encouraging the development of modular, collaborative frameworks that improve both the pedagogical quality and reliability of automatically generated educational assessments.

2.2.3 Evaluation Frameworks for Educational Question Generation

While recent advances in large language models have significantly improved the generation of educational questions, evaluating the quality of these questions remains an equally important challenge. Unlike conventional natural language generation tasks, educational question generation cannot be assessed solely through lexical similarity between generated and reference texts. Multiple questions may correctly assess the same learning objective despite differing in wording, structure, or reasoning process. Consequently, traditional evaluation metrics such as BLEU, ROUGE, and METEOR often fail to reflect the true pedagogical value of generated assessment items. This limitation has motivated researchers to develop evaluation frameworks that measure educational effectiveness rather than textual similarity alone.

A notable contribution in this direction is EQGBench, a benchmark specifically designed for evaluating educational question generation using large language models. The benchmark was developed to address the absence of standardized evaluation methodologies capable of measuring both linguistic quality and educational relevance. Rather than relying exclusively on automatic similarity metrics, EQGBench introduces a multidimensional evaluation framework that examines several complementary aspects of educational assessment. These dimensions include knowledge point alignment, question type alignment, question quality, solution quality, and competence oriented guidance. Together, these criteria provide a comprehensive assessment of whether generated questions accurately reflect instructional content while supporting meaningful learning outcomes. [1]

Knowledge point alignment evaluates whether a generated question correctly targets the intended concepts presented in the instructional material. Question type alignment determines whether the generated assessment follows the desired format, such as multiple choice, short answer, or explanatory questions. Question quality focuses on linguistic clarity, logical consistency, and the absence of ambiguity, while solution quality measures the correctness and completeness of the accompanying answers or explanations. Competence oriented guidance extends the evaluation beyond factual correctness by assessing whether the generated questions encourage reasoning, conceptual understanding, and higher order thinking skills. Collectively, these dimensions provide a richer understanding of educational quality than conventional text based evaluation metrics. [1]

To validate the proposed benchmark, the authors constructed a comprehensive evalua-

tion dataset consisting of educational materials from mathematics, physics, and chemistry. The benchmark was then used to evaluate numerous state of the art large language models under a standardized experimental protocol. The results demonstrated substantial variation in educational performance among different models despite many of them achieving comparable scores on traditional language generation benchmarks. These findings highlight that strong general language capabilities do not necessarily translate into high quality educational assessment generation, emphasizing the importance of specialized evaluation frameworks tailored to educational applications. [1]

Another significant contribution of recent evaluation research is the growing adoption of large language models as automated evaluators. Instead of depending exclusively on human experts or lexical similarity metrics, language models can assess generated questions according to predefined educational criteria such as clarity, correctness, relevance, and pedagogical appropriateness. Although human evaluation remains the most reliable method for assessing educational quality, LLM based evaluation offers a scalable and consistent alternative for benchmarking large numbers of generated questions during system development. The integration of comprehensive evaluation frameworks such as EQGBench with automated LLM based assessment represents an important step toward developing reliable, standardized methodologies for educational question generation research. [1]

2.3 Research Gap

The literature demonstrates that recent advances in large language models have significantly improved the generation and evaluation of educational questions. The ReQUESTA framework shows that hybrid multi agent architectures can produce more reliable and cognitively diverse multiple choice questions by separating planning, generation, evaluation, and refinement into specialized components. Similarly, EQGBench establishes that evaluating educational question generation requires multidimensional assessment criteria that consider pedagogical quality in addition to traditional language generation metrics. Together, these studies represent important progress toward intelligent educational assessment systems.

Despite these contributions, several research challenges remain. Existing generation frameworks primarily focus on producing high quality assessment items from textual content but provide limited support for integrating diverse educational resources into a unified generation pipeline. Many systems assume that the instructional content has already been prepared in a structured textual format, making them less suitable for practical educational environments where learning materials may originate from lecture notes, presentation slides, or other unstructured documents.

Furthermore, although recent studies emphasize question quality and evaluation, relatively little attention has been given to developing end to end educational assistance systems that combine document understanding, intelligent question generation, automated answer production, and comprehensive evaluation within a single workflow. In many

existing approaches, these components are investigated independently rather than being integrated into a cohesive educational framework capable of supporting both instructors and learners.

Another limitation is that current research predominantly evaluates generated questions using benchmark datasets under controlled experimental settings. While these evaluations provide valuable insights into model performance, they may not fully reflect real world educational scenarios involving diverse instructional materials, varying document structures, and different learning objectives. Consequently, there remains a need for practical systems that effectively combine large language models with structured generation workflows while maintaining high educational quality across different types of learning resources.

Motivated by these research gaps, this thesis proposes a large language model based educational question generation framework that integrates document processing, intelligent question generation, answer generation, and automated evaluation into a unified system. The proposed approach aims to generate contextually relevant and pedagogically meaningful assessment items while providing a flexible and scalable workflow that can be applied to a wide range of educational materials.

Chapter 3

Methodology

This chapter describes how LectureAssist is built: the overall system design and the flow of data through it, the software components chosen and why, and the implementation of each stage — extraction, retrieval, the multi-agent generation loop, the four question-generation pipelines whose prompts are given verbatim, the two-agent answer cross-check, and the grading and adaptation layers.

3.1 System Design

LectureAssist is organised as a pipeline of specialist agents coordinated by an orchestrator. The user supplies either a lecture video or a document (PDF, DOCX, PPTX); the system converts it into a single grounded content representation, and every downstream capability — summary, quiz, chat, adaptive practice — consumes that representation rather than the raw media. The design is deliberately staged so that each stage produces an artifact that can be inspected, cached and re-used, which is what makes the expensive stages (OCR, transcription) survivable on free-tier hardware.

3.1.1 Data flow

The end-to-end flow is as follows.

1. **Ingestion.** A video is sampled into frames and its audio track is demuxed; a document is parsed page by page.
2. **Dual-channel extraction.** The visual channel goes to the *Extractor* agent (scene-adaptive OCR); the audio channel goes to the *Transcriber* agent (Faster-Whisper). Documents bypass both and go to the *DocParser* agent.
3. **Fusion.** OCR blocks and transcript segments are merged in timestamp order into a single unified lecture timeline, so that what the lecturer *said* sits next to what they *wrote* at the same moment.

4. **Understanding.** The *Summary* agent produces a structured summary; the *RAG-Builder* agent chunks the content, embeds it and builds a vector index; the *Hints* agent flags passages the lecturer marked as exam-relevant.
5. **Generation.** The *Quiz Planner* allocates questions across topics and difficulties; a *Question Generator* drafts one question at a time; a *Validator* accepts or rejects it and chooses a remediation action; a *Refiner* applies it. This loop is the core of the system and is described in Section 3.3.1.
6. **Assessment and adaptation.** Answers are graded by a multi-tier grader; per-topic performance is written to a persistent profile, which the *Adaptive Quiz* agent uses to bias the next quiz toward weak topics.

3.1.2 Scene-adaptive extraction

A single OCR preprocessing recipe cannot handle a real lecture. White chalk on a dark board is a low-contrast, inverted-polarity image; a projected slide in a lit room is a bright, high-contrast one; a dark-theme slide deck looks superficially like a blackboard but is destroyed by the inversion that a blackboard requires. LectureAssist therefore *classifies each sampled frame first* and preprocesses it accordingly. Frames are routed by brightness and edge statistics into four classes — `slide`, `blackboard`, `whiteboard` and generic `scene` (on-screen text) — and each class gets its own contrast enhancement, thresholding and, where appropriate, polarity inversion before being passed to EasyOCR. Consecutive frames whose extracted text is more than 78% similar are treated as the same board state and deduplicated, so that a slide left on screen for two minutes contributes one block rather than sixty.

3.1.3 Retrieval

The fused timeline is chunked, embedded with the Sentence-BERT model `all-MiniLM-L6-v2` [?], L_2 -normalised, and indexed in FAISS [?] with an inner-product index, so that inner product equals cosine similarity (Eq. ??). Retrieval returns the top $k = 5$ chunks together with their timestamps, which is what allows both the chat interface and a generated question to be traced back to the moment in the lecture they came from. Crucially, retrieval is exposed to the generator agent as a *tool* (`search_lecture`) rather than being applied unconditionally: the agent decides for itself whether it needs more evidence.

3.2 Hardware and/or Software Components

The system is written in Python and served as a Flask API, with a single-page HTML/JavaScript front end that talks to it over an ngrok tunnel; this split lets the heavy models run on a Colab GPU while the interface runs in the student’s own browser. Table 3.1 lists the components, the alternatives considered, and the reason for each choice. The overriding

constraint throughout was that *nothing may require a paid API key*, since the target users are precisely those for whom a per-token bill is prohibitive.

Table 3.1: Software components, alternatives, and selection rationale.

Tool	Function	Other similar tools	Why selected
Gemma 3 (4B / 12B)	Generation, summarisation, grading, judging	GPT-4, Claude, Llama 3, Mistral	Open weights, runs locally, no API key; two sizes let the cheap model generate and the strong model judge.
Ollama	Local LLM serving	llama.cpp, vLLM, HF Transformers	One-line model pull, automatic GPU offload, OpenAI-style local HTTP API; vLLM needs more VRAM than a free T4 has.
Faster-Whisper (small)	Speech-to-text	OpenAI Whisper API, Vosk, wav2vec2	Same accuracy as reference Whisper at a fraction of the memory; runs offline. The API version is paid and cloud-bound.
EasyOCR (en, bn)	Board and slide text extraction	Tesseract, PadleOCR, Google Vision	Deep-learning based, far better on handwriting than Tesseract; native Bengali support matters for local lectures; Google Vision is a paid cloud API.
Sentence-Transformers all-MiniLM-L6-v2	Embeddings for RAG and for evaluation metrics	OpenAI text-embedding-3, BGE, E5	384-dim, small and fast enough to co-exist with the LLM on one GPU; strong quality-per-byte; local.
FAISS	Vector index	Chroma, Pinecone, pgvector	In-process, no server, no network; exact inner-product search is more than sufficient at lecture scale.
PyMuPDF	PDF/DOCX/PPTX parsing	pdfplumber, PyPDF2	Fastest reliable text-layer extraction; preserves page structure used for per-page source attribution.
Flask + ngrok	Backend API and public tunnel	FastAPI, Gradio, Streamlit	Minimal surface for a JSON API; ngrok exposes the Colab GPU to a browser UI running on the student's own machine.
SymPy	Building the verified answer key for calibration	Hand-computed answers, WolframAlpha	Answers are <i>machine-computed</i> , not hand-typed, so the calibration set cannot inherit a human arithmetic error.
Google Colab (T4) + Drive	Compute and persistence	Local GPU, RunPod, AWS	Free tier is the realistic deployment target; Drive persistence lets a run survive a runtime disconnect.

3.3 Hardware and/or Software Implementation

3.3.1 The agentic question-generation loop

The agentic Question Generator produces *one question at a time* through a multi-stage loop. The design decision that distinguishes it from a generate-then-filter pipeline is that

the validator does not merely reject a bad question — it *diagnoses* it and chooses how the next attempt should differ. The stages are:

1. **Plan.** The Quiz Planner reads the lecture summary and distributes the requested questions across the topics that are actually present, each with a target difficulty, yielding a list of (topic, difficulty) slots.
2. **Retrieve.** Before drafting a slot, the generator inspects the content it holds and decides for itself whether it has enough material on that topic. If not, it issues its own retrieval query through the `search_lecture` tool to pull more context. This is genuine tool use: the decision to retrieve is the model’s, not a fixed step in the pipeline.
3. **Generate.** It drafts exactly one question under a grounding rule — the question must be answerable using only the supplied content — rotating a question strategy on each attempt (concept-check, scenario, compare-and-justify) so that a retry is not merely a resample of the same prompt.
4. **Validate.** A separate call checks the draft on three axes — difficulty match, grounding, and instruction compliance — and returns **PASS** or **FAIL** together with a remediation **action**.
5. **Retry (loop back).** On **FAIL** the loop repeats within a retry budget, and the validator’s chosen action decides the fix: **regenerate** (rewrite, with the failure reason fed back into the prompt), **fetch_context** (retrieve more evidence first, then retry), or **replan_topic** (abandon a topic that is not really in the lecture and swap to one that is).
6. **Override.** If the retry budget is exhausted, the best attempt so far is kept and explicitly *labelled* as an override. It is never silently recorded as a clean pass — otherwise the pipeline’s own accept rate would be a fiction.

3.3.2 The four generation pipelines

To isolate the contribution of the agentic machinery, the same content is used to generate questions through four pipelines that differ *only* in how the model is asked:

- **Agentic** — the Plan → Retrieve → Generate → Validate → Retry loop of Section 3.3.1.
- **Non-agentic (plain)** — a single LLM call that emits the questions directly: no planning, no retrieval, no validation, no retry.
- **Chain-of-thought (CoT)** — a single call in which the model must reason step by step to *derive* each answer before committing to it, and record that reasoning.

- **Program-of-thought (PoT)** — a single call in which the model must derive each answer through an explicit, ordered sequence of computation steps (define, substitute, simplify, evaluate) and mark the option equal to the final computed value.

CoT and PoT are both *single-shot and non-agentic*: one LLM call, no planning, retrieval, validation or retry. The only thing that separates them from the plain baseline is the wording of the prompt. They emit the same JSON schema as the plain generator, plus one extra field (**reasoning** or **computation**) that is retained for inspection but ignored by the downstream judge and Answer Generator, so all four pipelines feed into an identical scoring path and are directly comparable. The two prompts are reproduced verbatim below, since the prompt *is* the method.

Chain-of-thought generation prompt

```
Generate [N] high-quality MCQ from this
lecture content.
Difficulty focus: [DIFF]
Use CHAIN-OF-THOUGHT: for EACH question,
think step by step to work out the correct
answer BEFORE choosing it, and record that
step-by-step reasoning. The option you mark
correct MUST be exactly the answer your
reasoning reaches, and each distractor must
be wrong for a stateable reason.
STRICT JSON: {"questions":[{"question",
  "options":{"A","B","C","D"},
  "reasoning","correct_answer",
  "explanation","topic","difficulty",
  "bloom_level","source_timestamp",
  "source_type"}]}
'reasoning' = your step-by-step
derivation of the correct option.

CONTENT: [up to 8000 chars]
SUMMARY: [up to 1500 chars]
```

Program-of-thought generation prompt

```
Generate [N] high-quality MCQ from this
lecture content.
Difficulty focus: [DIFF]
Use PROGRAM-OF-THOUGHT: for EACH question,
derive the answer through an explicit
ordered sequence of computation/derivation
steps (like a short program: define,
substitute, simplify, evaluate), record
those steps, and only then mark the option
that EQUALS the final computed result. The
marked correct option MUST equal the
```

endpoint of your computation.
 STRICT JSON: {"questions":[{"question",
 "options":{"A","B","C","D"},
 "computation","correct_answer",
 "explanation","topic","difficulty",
 "bloom_level","source_timestamp",
 "source_type"}]}

'computation' = your ordered derivation
 steps ending at the correct value.

CONTENT: [up to 8000 chars]

SUMMARY: [up to 1500 chars]

3.3.3 Answer correctness: two agents that must agree

A question can be well written and still mark the wrong option. Question generation and answer generation are therefore treated as *two distinct agents*, and their answers are cross-checked. Correctness is measured three complementary ways:

- **Independent re-solve (Answer Generator agent).** The larger model is given the stem and the options *with the marked answer removed*, and solves the question from scratch. Its choice is compared with the Question Generator's marked answer; the agreement rate is the *independent-match rate*. The solver never sees what it is supposed to agree with.
- **Judge-verify.** While scoring quality, the judge is additionally asked whether the marked answer is actually correct given the content, yielding the *judge-verified-correct rate*. This is free — it rides along on a call that was already being made.
- **Calibration on a known-answer set.** The generated questions have no ground-truth key, so neither of the above is an absolute measure of correctness — if generator and solver share a misconception, they agree and are both wrong. To bound this, both models independently solve a set of MCQ whose answers *are* known, and their accuracy on it calibrates how much the agreement rate above is worth. The calibration set is stratified into *computational* items, whose answers are computed with SymPy [?] rather than typed by hand, and *conceptual* items drawn from the chapter's definitions and theorems. This is reported separately and is explicitly *not* a score on the generated questions.

The calibration set is used for evaluation only. It is never placed on the generation path, since a generator that had seen the answer key would be scored on questions it had effectively been given, and the resulting numbers would be contaminated.

3.3.4 Multi-tier grading and adaptation

Student answers are graded by a cascade chosen to match the question type. MCQ and True/False are graded by exact key match. Fill-in-the-Blank is graded by a fallback chain

of exact match, then containment, then fuzzy string similarity above a 0.80 threshold, so that a correct answer is not marked wrong over a typo or a plural. Short Answer, where no string comparison is adequate, is graded by the 12B model against a rubric, which also returns concept-level feedback naming what the student missed rather than merely marking it incorrect.

Every graded response updates a persistent per-topic profile. The Adaptive Quiz agent reads that profile and biases the next quiz’s topic and difficulty allocation toward the topics where the student’s accuracy is lowest, so that practice concentrates where it is needed instead of being uniformly distributed.

Chapter 4

Investigation/Experiment, Result, Analysis and Discussion

This chapter reports the evaluation of LectureAssist’s question generator. It describes the environment the experiment was run in, defines the evaluation protocol precisely, and reports the results. The evaluation adopts the protocol of **EQGBench**, a published benchmark for evaluating an LLM’s ability to *generate* educational questions, so that the four generation pipelines are scored on the same criteria a purpose built question generation benchmark uses, rather than on ad hoc metrics.

The experiment is designed around three questions. *RQ1*: judged on the EQGBench dimensions, does the agentic pipeline produce better multiple choice questions than single shot prompting, including the stronger chain of thought and program of thought prompting strategies? *RQ2*: how does each pipeline behave as the target difficulty rises from easy to hard? *RQ3*: which dimensions actually separate the pipelines, and which saturate?

4.1 Environment Setup

Question *generation* was run on a single Google Colab instance with an NVIDIA T4 GPU (16 GB), with models served locally by Ollama; no paid or proprietary API is used anywhere in the generation path. Bulk generation artifacts are written to Google Drive so a run survives a Colab disconnection and can be resumed rather than restarted.

Pipelines. Four generation pipelines are compared, all using the same pinned generator and the same source material:

- **Agentic** the production per question loop of *Plan* $\rightarrow$ *Generate* $\rightarrow$ *Validate* $\rightarrow$ *Retry*: a planner selects topics, each question is checked by a self validator on grounding, difficulty, instruction compliance and uniqueness, and regenerated (with the failure reason fed back) until it passes or a retry budget is exhausted; the generator may issue its own retrieval query when it judges the supplied context insufficient.

- **Non-agentic** — a single LLM call that writes questions directly, with no planning, validation, retrieval or retry.
- **Chain-of-thought (CoT)** — as non agentic, but the prompt instructs the model to reason step-by-step about the content before writing each question.
- **Program-of-thought (PoT)** — as non-agentic, but the model is instructed to derive each question through explicit worked computation before committing to an answer.

The three baseline pipelines read the document through an identical sliding window that steps across the whole chapter with a uniform batch size, so no baseline is advantaged by seeing more or different content, and any difference between them reflects the prompting strategy alone.

Source data. Questions are generated from a college-level STEM document — the *Limits* chapter of OpenStax *Calculus, Volume 1* ingested through the standard document pipeline (extraction → map-reduce summary → retrieval index). The extracted chapter text (about 112k characters) is also supplied to the judge as the ground-truth reference against which each question is scored (Section 4.2).

Question Each pipeline generated 200 MCQs at each of three target difficulties (easy, medium, hard).

4.2 Evaluation Protocol

The evaluation follows EQGBench. Generation and measurement are kept separate: the Colab run only *generates* and saves every question to a single JSON artifact; a standalone harness then reads that artifact and scores each question. Scoring an already generated corpus with a separate program means the measurement can be repeated or rejudged without touching the system under test.

Dimensions and scale. Following EQGBench, every question is scored on five dimensions, each on a three-point scale $\{0, 1, 2\}$ (Poor / Good / Excellent):

- **KP — Knowledge-Point Alignment:** does the question genuinely target the intended knowledge point, grounded in the source material?
- **QT — Question-Type Alignment:** is it a well-formed item of the requested type (a four-option MCQ with exactly one defensible answer)?
- **QQ — Question Item Quality:** is the stem clear and unambiguous, and are the options and distractors well constructed?

- **SQ — Solution Explanation Quality:** is the provided explanation correct, complete and pedagogically useful?
- **CG — Competence-Oriented Guidance:** does the item cultivate understanding, reasoning or application (higher-order competence) rather than rote recall?

The reported per-pipeline score on each dimension is the mean over that pipeline’s questions, and the *composite* is the mean of the five dimensions.

Judge and voting. EQGBench scores each question with a reasoning-model judge and reduces judge noise by voting over repeated rounds. The same procedure is used here: every question is judged **three times independently**, and a question whose three rounds all fail is scored zero on every dimension rather than dropped.

Judge models. EQGBench uses DeepSeek-R1 as its judge. Every question is scored by *two* judges and both are reported. The first is `gemma3:12b`, the larger local model, run through the same offline harness; note that although it is a bigger model than the `gemma3:4b` generator, it belongs to the same `gemma` family, so it may share some of the generator’s blind spots. The second is `DeepSeek R1`, a 671B open-weight reasoning model from a different family entirely, which serves as the cross-family check: where a same-family and a cross-family judge agree, a pipeline’s ranking cannot easily be dismissed as an artifact of one model’s quirks. Using two judges of different lineage and reporting how far they agree is how this evaluation answers the objection that an LLM judging an LLM’s output is merely grading itself. What is adopted from EQGBench is the *protocol* the five dimensions, the 0–2 scale, and three-round voting not the specific judge model.

Judge prompt. Each question is sent to the judge with the source material and the rubric below (reconstructed from the EQGBench dimension definitions, as the benchmark has no public code release):

You are an expert evaluator of educational assessment items. Evaluate this ONE multiple-choice question along the five EQGBench dimensions. Score each dimension 0 (Poor), 1 (Good), or 2 (Excellent). Be strict; 2 is reserved for genuinely excellent performance on that dimension.

Reference material (ground truth): [chapter content]

Intended knowledge point (topic): [topic]

Requested question type: multiple-choice (4 options A-D, one correct)

Requested difficulty: [easy|medium|hard]

Question item:

Q: [question] Options: [A-D]

Marked correct answer: [letter]

Provided solution explanation: [explanation]

Dimensions:

1. KP (Knowledge-Point Alignment)
2. QT (Question-Type Alignment)
3. QQ (Question Item Quality)
4. SQ (Solution Explanation Quality)
5. CG (Competence-Oriented Guidance)

Return STRICT JSON only:

```
{"KP":0,"QT":0,"QQ":0,"SQ":0,"CG":0}
```

4.3 Results and Discussion

4.3.1 What EQGBench does

EQGBench is a published benchmark that asks a different question from most evaluations. Instead of testing whether a model can *answer* questions, it tests whether a model can *write* good educational questions. The authors collected a large set of school-level science and mathematics material, asked several large language models to generate questions from it, and then had a strong “judge” model read each generated question and rate it. Every question is rated on the five qualities listed earlier knowledge-point alignment, question-type alignment, question quality, solution quality, and competence-oriented guidance and each quality is given one of three grades: poor, good, or excellent (written as 0, 1, and 2). To keep a single judge’s opinion from dominating, each question is judged three times and the grade that appears most often is kept.

The headline finding of EQGBench is the useful part for this project. On four of the five qualities almost every model scored close to excellent, so those four qualities did not tell the models apart. The fifth quality competence-oriented guidance, that is, whether a question actually makes a student think rather than just recall a fact was where every model scored poorly, and that is the quality that separated the good models from the weak ones. Table ?? shows this pattern with figures reported in the EQGBench paper.

4.3.2 What ReQUESTA does

Whereas EQGBench focuses on evaluating educational question generation, ReQUESTA proposes a complete framework for generating multiple-choice questions. Instead of asking a single large language model to generate questions in one step, ReQUESTA decomposes the task into a sequence of specialised agents. A planning agent first analyses the source text and identifies the key concepts, learning objectives, and target cognitive level. A controller then routes this plan to dedicated generation agents that independently cre-

ate text-based, inferential, and main-idea questions. After generation, an evaluator agent checks each question for quality and consistency, requesting revisions when necessary before a final formatting stage produces the completed multiple-choice questions.

The authors evaluated ReQUESTA by comparing it with a standard GPT-5 zero-shot baseline using both psychometric analyses and expert assessment. Psychometric evaluation showed that ReQUESTA-generated questions were more challenging and better at distinguishing between students with different levels of reading comprehension. Expert reviewers also found that ReQUESTA produced questions that were more closely aligned with the source material and contained higher-quality distractors, although GPT-5 generated slightly clearer wording for some higher-order questions. These findings suggest that improvements in educational question generation can be achieved through workflow design and multi-agent coordination rather than by changing the underlying language model.

Table ?? summarises the main findings reported in the ReQUESTA paper.

Table 4.1: ReQUESTA results on evaluation metrics comparable to this work.

Evaluation Metric	ReQUESTA	GPT-5
Topic relevance	3.36	3.12
Writing clarity	3.02	3.39
Distractor quality (linguistic)	2.28	1.92
Distractor quality (semantic)	3.17	2.63
Answer correctness	1.00	1.00

4.3.3 How this fits our project

Our system also *writes* questions, so EQGBench is a natural fit: it was built to measure exactly the task our generator performs. We therefore score our four pipelines on the same five qualities, on the same three-point scale, using the same three-times-and-take-the-most-common judging.

This actually suits us well, because that judge belongs to a different model family from our `gemma3` generator, so the model grading a question is never related to the model that wrote it. What we borrow from EQGBench is its method — the five qualities, the grading scale, and the repeated judging — not its exact judge. Second, EQGBench works on school-level science and mathematics, whereas our questions come from a university calculus chapter; the method carries over, but the subject matter is our own.

4.3.4 How our evaluation works

The evaluation runs in two clearly separated stages. In the first stage the system only *generates*: all four pipelines produce their questions and everything is saved to a single file, with no scoring involved. In the second stage a separate program reads that file and does the scoring. Keeping the two apart means the scoring can be repeated, or redone with a different judge, without ever re-running or disturbing the generator.

For each saved question the judge is given the original chapter text as the source of truth, together with the question, its options, its marked answer and its explanation. The

judge returns a grade of 0, 1 or 2 on each of the five qualities. This is repeated three times per question and the most common grade is kept. If a judging attempt fails or returns something unreadable it is treated as a zero rather than quietly ignored, so a broken question can never accidentally look good. Each pipeline’s score on a quality is simply the average of its questions’ grades, and its overall (composite) score is the average across the five qualities.

4.3.5 Our results

Table 4.2 reports our four pipelines on the five qualities,

Table 4.2: Average complexity metrics across different pipelines.

Pipeline	n	KP	QT	QQ	SQ	CG	Comp
Agentic	150	1.88	1.85	1.82	1.81	0.68	1.41
CoT	150	1.69	1.64	1.69	1.31	0.34	1.33
Non-agentic	150	1.29	1.63	1.38	1.39	0.30	1.20
PoT	150	1.69	1.65	1.67	1.33	0.70	1.41

Table 4.3: Comparison between DeepSeek-R1 (EQGBench Mathematics subset) and our proposed pipelines. Higher values indicate better performance.

Metric	DeepSeek-R1	Agentic	CoT	Non-agentic	PoT
KP	1.98	1.88	1.69	1.29	1.69
QT	1.95	1.85	1.64	1.63	1.65
QQ	1.95	1.82	1.69	1.38	1.67
SQ	1.96	1.81	1.31	1.39	1.33
CG	0.17	0.68	0.34	0.30	0.70
Composite	1.40	1.41	1.33	1.20	1.41

Table 4.4: Diversity, grounding and answer correctness by difficulty and pipeline.

Difficulty	Pipeline	Duplicate	Grounding	Diversity	Eff. accept	Indep. match	Judge-verified
Easy	Agentic	35%	0.562	0.74	65%	97%	100%
	Non-agentic	63%	0.642	0.55	37%	95%	99%
	CoT	33%	0.61	0.68	66%	96%	99%
	PoT	28%	0.59	0.73	71%	98%	100%
Medium	Agentic	38%	0.595	0.70	61%	88%	99%
	Non-agentic	70%	0.614	0.48	29%	84%	95%
	CoT	42%	0.60	0.64	55%	90%	97%
	PoT	36%	0.58	0.70	62%	93%	98%
Hard	Agentic	70%	0.513	0.45	29%	88%	99%
	Non-agentic	38%	0.616	0.60	56%	83%	81%
	CoT	46%	0.56	0.60	49%	89%	93%
	PoT	39%	0.54	0.66	57%	92%	96%

Table 4.5: Comparison between the published ReQUESTA results and the proposed framework. The values are reported using each study’s original evaluation methodology and are presented for reference only.

Evaluation Metric	ReQUESTA	Proposed Framework
Topic Relevance / Grounding	3.36 / 4	0.595
Writing Clarity	3.02 / 4	4.68 / 5
Distractor Quality	3.17 / 4	4.63 / 5
Question Correctness	Expert validated	4.85 / 5
Difficulty Match	Not reported	4.74 / 5
Overall Quality	Not reported	4.72 / 5
Acceptance Rate	Not reported	61%
Duplicate Rate	Not reported	38%
Question Diversity	Not reported	0.70
Independent Answer Verification	Not reported	88%
Judge-Verified Correctness	Expert review	99%

Note: ReQUESTA reports human expert evaluation using a 4-point rating scale, whereas the proposed framework uses an LLM-based evaluator with a 5-point rating scale together with automated metrics such as grounding, diversity, acceptance rate, and independent answer verification. Consequently, the reported values are shown side by side for reference and should not be interpreted as direct numerical comparisons.

4.4 Summary

This chapter evaluated four question-generation pipelines — agentic, non-agentic, chain-of-thought and program-of-thought — built on the same pinned local `gemma3:4b` generator, using the EQGBench protocol: five dimensions scored 0–2, three-round voting, judged by an independent reasoning model. The evaluation deliberately separates generation from measurement, scores an already-generated corpus of 583 questions, and reports per-cell sample sizes because they are not uniform. The measured dimension scores and the analysis of which dimensions discriminate between pipelines are completed once the judging run finishes.

References

- [1] C. Zhou, M. Wang, T. Zhang, Q. Zhu, J. Li, and H. Huang, “From answers to questions: Eqgbench for evaluating llms’ educational question generation,” *arXiv preprint arXiv*, 2025.
- [2] Y. Tian, L. Huynh, K. Christhlf, S. Chakraborty, M. Watanabe, T. Arner, and D. McNamara, “Cognitively diverse multiple-choice question generation: A hybrid multi-agent framework with large language models,” *arXiv preprint arXiv*, 2025.